Year II — Computational Mathematics

Based on lectures by Various Lecturers

Notes taken by Anonymous

Michaelmas 2026

Contents

1 Numerical and Symbolic Computing **2**
1.1 Fixed-point numbers 2
1.2 Floating-point numbers 2

Chapter 1

Numerical and Symbolic Computing

1.1 Fixed-point numbers

We see that when we attempt $0.1 + 0.2 == 0.3$ in MATLAB, we return a false value due to rounding in binary causing issues. Numbers like 0.1 and 0.2 cannot be stored in binary exactly and thus the two sides will differ by 5.6×10^{-17}. We utilise real numbers and integers within computational systems to speed up manipulating mathematical objects. We can do this symbolically with *Computer Algebra Systems* but we trade off time.

Definition 1.1.1 (Fixed-point numbers)**.** A *fixed-point number* representation of a real number is given as:

$$x = \pm \left(d_1 d_2 \cdots d_{k-1} \cdot d_k \cdots d_n\right)_\beta, \qquad \text{where} \qquad d_1, \ldots, d_n \in \{0, 1, \ldots, \beta - 1\}.$$

Here β is the base (e.g. 10 for decimal arithmetic or 2 for binary). The word fixed refers to the position of the point: a fixed-point system uses a fixed number of digits before and after the point, so the numbers it can represent are equally spaced, a distance $\beta^{-(n-k+1)}$ apart.

Remark 1.1.2. This system can be used to represent any real number uniquely. We must remove representations ending in infinitely many 9's (when in decimal) or infinitely many $b-1$'s where b represents the base. For example in denary we cannot use $3.14 = 3.1400\ldots = 3.139999\ldots$ as we are excluding the $10 - 1$'s expansion.

1.2 Floating-point numbers

Definition 1.2.1 (Floating-point numbers)**.** Fix a base $\beta \geq 2$, a number of digits m, and an exponent range $e_{\min} \leq e \leq e_{\max}$. Only numbers in the floating-point number system $F \subset \mathbb{R}$ can be represented exactly, where

$$F = \left\{ \pm \left(0.d_1 d_2 \cdots d_m\right)_\beta \beta^e \mid d_i \in \{0, 1, \ldots, \beta - 1\}, e \in \mathbb{Z}, e_{\min} \leq e \leq e_{\max} \right\}$$

Here $\left(0.d_1 d_2 \cdots d_m\right)_\beta$ is called the fraction (or significand or mantissa), and e is the exponent.

Example 1.2.2. Suppose we set $\beta = 10$ and $m = 3$ and:

$$-2 \leq e \leq 2.$$

Then we could represent

$$0.123 \times 10^2 = 12.3$$

or

$$0.456 \times 10^{-2} = 0.00456.$$

We could also represent

$$-0.789 \times 10^1 = -7.89.$$

But something like

$$12.3456$$

cannot be represented exactly, because we only have 3 digits available for the significand.

Example 1.2.3. Now consider a system with $\beta = 2$ so we are working in binary. We set $m = 3$ and once more we have:

$$-2 \leq e \leq 2.$$

We now represent numbers such as:

$$(0.101)_2 2^2 = (\frac{1}{2} + \frac{1}{8}) \times 4 = \frac{5}{8} \times 4 = 2.5$$

or something like

$$(0.100)_2 2^1 = (\frac{1}{8}) \times 2 = 0.25$$

Definition 1.2.4 (Normalisation)**.** We use normalisation, enforcing the requirement $d_1 \neq 0$ to create a unique numbering system and so we assume that our system is *normalised* by default. This is useful as we can represent a larger range of numbers at the loss of equally spaced numbers.